Tensor Networks for Efficient Representation and Compression of Large Language Models

Ruwail Hassan
University College London

Abstract

Large Language Models (LLMs) have achieved state-of-the-art performance through massive parameter scaling, but their increasing training and deployment costs have raised scalability concerns. Conventional compression techniques such as pruning and quantisation reduce model size by removing or approximating parameters while leaving the underlying representational structure unchanged, limiting their effectiveness under aggressive compression. This review examines tensor networks (TNs) as a mathematically principled alternative, originating from quantum many-body physics as a means of efficiently representing high-dimensional wavefunctions. Empirical evidence suggests that LLM weights exhibit low-rank structure and substantial redundancy; TNs exploit this by reparametrising weight matrices as networks of smaller tensors connected by bounded bond dimension, explicitly constraining correlations. This structural reparametrisation results in a more compact and efficient representation that aligns with the intrinsic dimensionality of the learned weights. We trace the development of TN-based compression from classical neural networks to modern transformer architectures, finding that TN decompositions achieve compression ratios beyond conventional methods while preserving or improving performance and can be combined with standard techniques like quantisation for further gains. We discuss practical barriers to adoption, including the theoretical efficiency of TNs not being realised practically due to current GPU hardware optimised for dense matrix multiplication. Our findings suggest that realising the full potential of TN compression requires co-design across hardware and software.

CONTENTS

1 Introduction **3**

2 Origins and Motivation **3**

3 Mathematical Background **4**

3.1 Graphical Notation and Contractions 4

3.2 Matricisation and SVD 5

3.3 Bond Dimension 5

3.4 Tucker decomposition 5

3.5 Matrix Product State / Tensor Train 5

4 Tensor Networks in Classical Neural Networks **6**

5 Tensor Networks in Transformers and Large Language Models **7**

6 Discussion **8**

7 Conclusion **9**

1 INTRODUCTION

Large Language Models (LLMs) underpin recent advances in natural language processing, multimodal reasoning and autonomous decision making. Their state-of-the-art performance has largely been achieved through brute-force scaling, with frontier models containing trillions of parameters. Due to their huge size, they require vast computational resources to train, store and deploy. Training costs for frontier models are projected to exceed one billion dollars by 2027 [1], while inference has emerged as a dominant cost as providers spend billions annually serving models at scale. As LLMs become increasingly embedded in real world systems, these costs raise serious sustainability concerns.

Model development has been guided by empirical scaling laws, which demonstrate predictable performance gains with increased model size, data and compute [2]. This perspective was reinforced by Google's PaLM (540B) [3] which seemingly demonstrated that capabilities such as chain-of-thought reasoning were 'emergent' properties unlocked only at massive scale. Consequently, the brute-force scaling strategy dominated; while effective, its long-term viability is uncertain. Classical scaling laws implicitly assume access to unlimited high-quality data, an assumption increasingly violated in practice, and empirical evidence suggests diminishing returns from further parameter increase. For example, Llama-2-70B retains near baseline reasoning performance even after pruning approximately 50% of neurons in its deepest layers, indicating substantial redundancy [4].

Existing compression techniques such as pruning and quantisation reduce model size by removing parameters or lowering their numerical precision. They operate primarily as post-training optimisations that leave the underlying representational structure unchanged. As a result, aggressive compression can impair multi-step reasoning even when average accuracy remains high.

This review explores tensor networks (TNs) as an alternative approach to efficiency in large-scale machine learning. TNs impose explicit hierarchical structure and correlation constraints on high-dimensional tensors, offering a principled reparametrisation of neural network weights. By treating compression as a structural prior rather than a post-hoc optimisation, TNs provide a pathway toward smaller, more efficient models and suggest a promising direction for improving reasoning efficiency beyond brute-force scaling.

2 ORIGINS AND MOTIVATION

TNs originate in quantum many-body physics as a means of efficiently representing high-dimensional wavefunctions. A quantum system of N particles is described by a wavefunction in a Hilbert space of dimension d^N , yet physically realisable states occupy a highly constrained subset characterised by limited entanglement [5]. Tensor networks exploit this structure by approximating exponentially large tensors through structured factorisations that retain only the correlations required to represent the underlying state.

At a representational level, tensor networks express high-dimensional tensors as a network of smaller tensors connected by contracted indices, which explicitly encode correlations. The expressive capacity of a tensor network is governed by its bond dimensions, which control the amount of statistical dependence that can be represented between different tensor partitions.

This perspective naturally extends to machine learning, where dense weight matrices are often large and highly redundant. Modern neural networks are intentionally overparametrised, which enables effective optimisation but results in a large, slow model. Dense layers allow every output dimension to depend on every input dimension, often resulting in weight matrices exhibiting strong internal correlations suggesting they have a lower effective rank.

This is supported by the "Universal Weight Subspace Hypothesis" [6] which shows that despite massive parameter count of frontier models their learned representations converge to low-dimensional manifolds. Spectral analysis of weights across architectures reveals a sharp spectral decay, indicating that the dominant variance is captured by a small number of weight directions. Consequently, higher order spectral components often represent noise rather than signal [7].

The current dominant application of tensor networks in machine learning is as a post-training reparametrisation of dense weights. Large pretrained weight matrices are decomposed into struc-

tured networks of smaller tensors, reframing compression as a change of representation rather than parameter removal. This introduces an implicit inductive bias: correlations are constrained by the bond dimension, favouring hierarchical and localised dependency structure. Low bond dimensions enforce aggressive compression by limiting statistical dependence, while high bond dimensions progressively recover the expressive capacity of the original dense representation, yielding a smooth trade-off between compression and accuracy. Tensor networks do not just compress; they align the model's structure with the intrinsic dimensionality of the data.

3 Mathematical Background

In practice, a tensor is often treated as a multi-dimensional array. Formally, an order N tensor X is an element of the tensor product of N vector spaces:

$$X \in V_1 \otimes V_2 \otimes \cdots \otimes V_n \cong \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_n}$$

This definition highlights the tensor's true definition as a multilinear map. Just as a matrix (second-order tensor) represents a linear map from one vector space to another, a higher-order tensor represents a map involving multiple spaces. The number of dimensions, N, is the order of the tensor.

A central challenge in high-dimensional data analysis is the curse of dimensionality. A generic tensor X requires $\prod_{n=1}^{N} I_n$ scalars for storage. For a uniform dimension I, this scales as $\mathcal{O}(I^N)$, making storage and manipulation exponentially expensive. Tensor networks (TNs) address this by factorising the global tensor into a set of smaller, low-order tensors contracted together, reducing the complexity to polynomial.

3.1 Graphical Notation and Contractions

To manage the complexity of indices, we utilise Penrose graphical notation; where nodes represent tensors and legs represent indices (vector spaces). Scalars, vectors, matrices and nth-order tensors correspond to nodes with zero, one, two or n legs respectively.

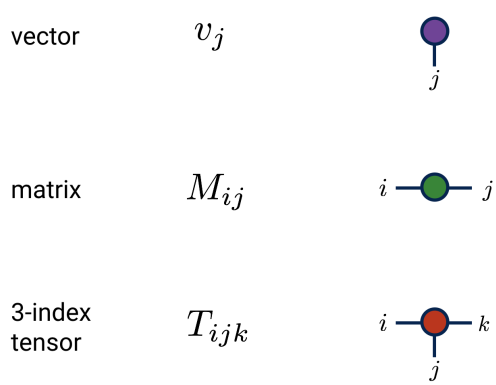

Figure 1: Penrose notation. Reproduced from [8]

Tensor contraction is the fundamental operation of TNs, generalising matrix multiplication. Algebraically, contracting two tensors A and B over a shared index k corresponds to the Einstein summation convention:

$$C_{i \cdots j} = \sum_{k=1}^{R} A_{i \cdots k} \, B_{k \cdots j}.$$

Graphically this is represented by connecting the legs of the two nodes. The index k becomes a 'bound' or internal edge, summed over while the uncontracted indices remain free.

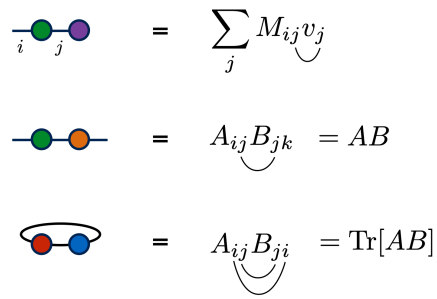

Figure 2: Tensor contractions. Reproduced from [8]

3.2 Matricisation and SVD

Tensor network decompositions are constructed via matricisation (unfolding), which reshapes a tensor into a matrix. The mode-n matricisation of a tensor X, denoted X_n, involves grouping the nth index as the row index and all other indices as columns.

This mapping enables the application of Singular Value Decomposition (SVD). For a matricised tensor X_n, the SVD yields:

$$X_n = U \Sigma V^T \approx U_r \Sigma_r V_r^T$$

Where retaining the largest r singular values yields the optimal low-rank approximation. The rank r defines the effective dimensionality of correlations across the corresponding tensor partition.

3.3 Bond Dimension

In a tensor network, the bond dimension χ corresponds to the rank of matricisation across the 'cut' represented by an edge. Intuitively χ acts as an information bottleneck, quantifying the amount of correlation (or entanglement in physics) that can be represented between the two subnetworks. A higher χ allows the network to capture more complex correlations but increases computational cost, typically $\mathcal{O}(\chi^3)$ for contractions; a lower χ enforces a more aggressive compression.

3.4 Tucker decomposition

The Tucker decomposition factorises a tensor into a central core tensor G interacting with a set of factor matrices $U^{(n)}$ along each mode. For a third-order tensor:

$$X_{ijk} \approx \sum_{p=1}^{R_1} \sum_{q=1}^{R_2} \sum_{r=1}^{R_3} G_{pqr}\, U^{(1)}_{ip}\, U^{(2)}_{jq}\, U^{(3)}_{kr}.$$

The factor matrices define low-dimensional subspaces along each mode, while the core tensor captures their interactions. Although Tucker effectively reduces individual mode dimensions, the core tensor remains Nth-order and scales as $\mathcal{O}(R^N)$, limiting scalability.

3.5 Matrix Product State / Tensor Train

The Matrix Product State (MPS) / Tensor Train (TT) decomposition is the most widely used due to its simplicity, numerical stability and linear scaling. TT decomposes the tensor into a linear chain of third-order tensors.

$$X_{i_1 i_2 \cdots i_N} = \sum_{\alpha_1, \ldots, \alpha_{N-1}} G^{(1)}_{i_1 \alpha_1} G^{(2)}_{\alpha_1 i_2 \alpha_2} \cdots G^{(N)}_{\alpha_{N-1} i_N}.$$

where, i_n are the physical indices and α_k are the virtual bond indices.

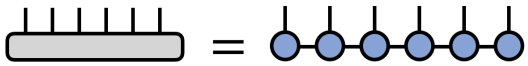

Figure 3: TT decomposition. Reproduced from [8]

The TT format is constructed via sequential SVD. One essentially sweeps through the tensor:

1. Matricise the first dimension separate from the remaining $N-1$ dimensions.
2. Apply SVD. The left singular vectors form the first core $G^{(1)}$.
3. Contract the singular values with the right singular vectors
4. Reshape the result into a residual tensor
5. Repeat for the next dimension.

This structure reduces the storage complexity from exponential $\mathcal{O}(I^N)$ to linear $\mathcal{O}(NI\chi^2)$

4 Tensor Networks in Classical Neural Networks

An early and influential example of tensor network reparametrisation in deep learning is the work of Novikov et al. [9], which reparametrised fully connected neural network layers as a sequence of TT cores and trained them end-to-end using standard backpropagation-based gradient descent.

Consider a fully connected neural network layer:

$$y = Wx + b, \quad W \in \mathbb{R}^{N\times M},\ x \in \mathbb{R}^M,\ y \in \mathbb{R}^N,\ b \in \mathbb{R}^N.$$

The TT approach reinterprets the flat matrix W as a higher-order tensor by factorising the input and output dimensions as:

$$M = \prod_{k=1}^{d} m_k, \quad N = \prod_{k=1}^{d} n_k.$$

This introduces a multi-index structure on the input and output dimensions,

$$j \leftrightarrow (j_1, \ldots, j_d), \quad i \leftrightarrow (i_1, \ldots, i_d),$$

allowing the weight matrix to be reshaped into a second-order tensor

$$W(i,j) \ \mapsto\ W(i_1, \ldots, i_d, j_1, \ldots, j_d), \quad W \in \mathbb{R}^{N\times M} \simeq \mathbb{R}^{n_1\times\cdots\times n_d\times m_1\times\cdots\times m_d}.$$

The resulting tensor is then represented using a TT factorisation

$$W(i_1, \ldots, i_d, j_1, \ldots, j_d) = \sum_{\alpha_1,\ldots,\alpha_{d-1}} \prod_{k=1}^{d} G^{(k)}(\alpha_{k-1}, i_k, j_k, \alpha_k),$$

where $G^{(k)} \in \mathbb{R}^{r_{k-1}\times n_k\times m_k\times r_k}$ are the Tensor Train cores and r_k denote the TT rank, with boundary conditions $r_0 = r_d = 1$. This decomposition constrains the weight tensor to exhibit bounded rank across successive unfoldings, imposing a hierarchical low-rank structure on the parameter space.

The Tensor Train ranks r_k act as the primary capacity control of the model: increasing the ranks increases expressivity, while lower ranks impose stronger compression and regularisation. Unlike classical low-rank matrix factorization, which enforces a single global rank constraint, the TT decomposition imposes a sequence of local rank constraints across tensor modes. The approach is effective for large linear layers as the parameter scales as $\mathcal{O}\Big(\sum_{k=1}^{d} r_{k-1}\, m_k\, n_k\, r_k\Big)$, rather than $\mathcal{O}(MN)$.

Empirically, Novikov et al. [9] showed that individual fully connected layers could be compressed up to $200,000\times$ with minimal accuracy loss and faster inference time. On ImageNet-scale VGG models, compressing fully connected layers resulted in an overall network compression of approximately $7\times$, with remaining parameters dominated by convolutional layers.

This work demonstrated that over-parametrisation in dense layers can be replaced by structured representations that preserve the learned function while constraining complexity. More broadly, it introduced the idea that compression itself can act as an inductive bias, shaping the class of functions a network can represent.

Subsequent work extended tensorisation beyond fully connected layers by demonstrating that Tensor Train decompositions could be applied to convolutional kernels by reformulating convolution as matrix multiplication before tensorisation [10]. They achieved $80\times$ network compression rate with only a 1.1% accuracy loss on CIFAR-10 dataset [10].

Together, these works establish the viability of tensor networks within classical neural network architectures, motivating their later adoption in transformer-based models.

5 Tensor Networks in Transformers and Large Language Models

Transformers are the dominant architecture behind Large Language Models, primarily consisting of multi-head self-attention (MHA) blocks followed by feed-forward networks (FFN) [11]. The MHA block transforms input token representations (vector encodings of subwords) into context-dependent representations, with multiple independent heads each learning distinct patterns over the same input. This is done by projecting the inputs through learned weight matrices (W_Q, W_K, W_V) into query (what a token seeks), key (what it possesses), value (its actual content) spaces. Attention weights are then computed via SoftMax-normalised dot-product between queries and keys, which is used to form a weighted sum of values, allowing each token to incorporate information from other relevant tokens in the sequence. The FFN then applies a position-wise non-linear transformation. Both components contain large dense weight matrices, while exhibiting structurally distinct redundancy patterns, which motivates decomposing them separately.

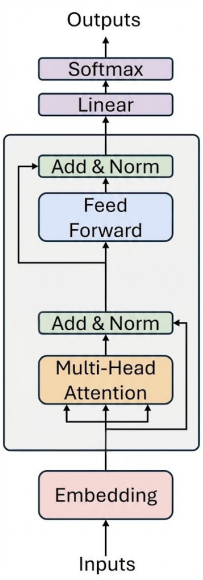

Figure 4: Transformer architecture. Usually consists of multiple FFN and MHA blocks. Adapted from [12]

Modern transformers are massively overparametrised. Although this improves optimisation, excess parameters can introduce noise and hinder reasoning. LASER [7] provides empirical evidence of this by showing that selectively removing higher-order spectral components of transformer weights improves reasoning performance, indicating that large models contain redundant structure that can impair generalisation.

Early attempts to address this redundancy through architectural redesign include the tensorised transformer [13]. They showed that the nonlinear SoftMax-based attention mechanism could be reformulated multi-linearly, allowing tensor factorisation of the attention operator via Block-Term Decomposition. Their tensor-native architecture achieved competitive language modelling performance with significantly fewer parameters than standard transformers of the time. Despite these results, the field largely shifted toward post-training compression methods, partly due to the strong alignment between conventional transformer operations and highly optimised dense matrix multiplication kernels on modern GPU hardware.

Further empirical evidence of redundancy in GPT-style models is provided by TensorGPT [14], a training-free compression method that applies TT to token embeddings. By reshaping individual embeddings into higher-order tensors then performing TT decomposition, the authors demonstrate significant compressibility of the embedding layer, with theoretical compression ratios 39–$65\times$ and practical reductions around $2\times$ with minimal performance loss. These results indicate low-rank structure in GPT family models.

While most methods decompose individual weight matrices, TRAWL [15] stacks weights from consecutive layers into a higher order tensor. It then applies CP, a rank-one tensor factorisation, or Tucker decomposition, effectively enforcing a shared feature basis across layers. The authors applied this to weights and performed no extra fine-tuning. They found that earlier layers and the MHA layers showed little to no benefit and decreased accuracy in some cases. However, the FFNs displayed high redundancy particularly in deeper blocks achieving performance improvements up to 16% over dense baselines. This paper confirms that FFNs are smooth and repetitive, however the attention mechanism is structurally distinct and requires a more complex geometry to capture inter-head correlations without degrading performance.

TensorLLM [12] addresses the limitations of TRAWL [15], by leveraging the intuition that attention heads within the same layer capture patterns at similar abstraction levels and operate in a shared subspace. Instead of storing the W_Q, W_K, W_V, W_O as large concatenated matrices, it splits them into per-head submatrices, stacks the four submatrices of each head into a third-order tensor and then stacks these across heads to form a fourth-order tensor. By applying Tucker decomposition, a shared subspace is enforced as all heads are constrained to the same set of factor matrices, while each head's specialised information is captured in a small head-specific core tensor. This method effectively structurally denoises the model resulting in TensorLLM achieving up to $250\times$ compression of the MHA block while consistently improving reasoning across benchmarks.

CompactifAI [16] demonstrates that TNs are a viable production-level compression framework. By using Matrix Product Operators (MPO), essentially 1D chains of tensors, combined with quantisation and a brief retraining phase they achieved a 93% memory reduction on Llama 2-7B with only 2-3% accuracy loss on standard benchmarks. Crucially, they showed that TN decomposition is complementary to standard compression methods like quantisation. This structural compression also translated to hardware gains, with a 50% distributed training speed-up and 25% faster inference due to reduced CPU-GPU data transfer overhead.

6 Discussion

Tensor decomposition operates in a fundamentally different space than conventional compression. Reshaping a weight matrix into a higher-order tensor exposes correlations across multiple axes of its new geometry, captured by its bond dimensions. Bond dimensions are distinct from the matrix rank; a full-rank matrix may still be accurately represented by a TN with low bond dimensions if its weights are strongly correlated in its reshaped geometry. Tensor decompositions exploit this hidden multilinear structure, achieving compression rates that standard methods cannot [17]. By enforcing a low bond dimension, TNs impose an inductive bias prioritising the dominant correlations within the weight tensor, discarding noisy higher-order components. This is theoretically grounded in the observation that real-world data is highly structured and gradient descent tends to align weight structure with data structure, suggesting well-trained models should naturally have low bond dimensions [17]. Thus, TNs should not be viewed as merely compression, but as structural regularisers enforcing a more compact representation of the learned function.

Despite the theoretical appeal, TNs have not replaced dense architectures. This is partly explained by the 'Hardware Lottery' [18]: successful methods are often those that fit the dominant hardware, not those that are mathematically superior. Modern GPUs are aggressively optimised for dense matrix-matrix multiplication, relying on contiguous memory blocks to saturate tensor cores. In contrast, TNs involve complex contractions that require frequent index permutations and reshuffles. These operations create significant memory movement overhead. Consequently, a tensorised model with far fewer FLOPs often runs slower than a dense model as the hardware spends more time moving data than computing it. This mismatch significantly limited the immediate scalability of tensorised architectures, forcing the field towards post-training compression.

The tensor network software stack also remains far less mature than its dense counterpart. Most frameworks rely on generic einsum operations that do not exploit the specific structure of tensor contractions, resulting in suboptimal memory access patterns and redundant data movement. Fusing index permutations with contraction steps into specialised kernels could significantly improve this overhead. Minima [19] demonstrates early progress by writing custom CUDA/Triton kernels for their production compression pipeline increasing throughput on a TN-compressed Qwen3-32B model. However, software optimisation alone is unlikely to close the gap entirely; TNs may require dedicated hardware optimised for high-order tensor operations to compete with the dense scaling paradigm fuelled by highly optimised GPUs.

TNs also face intrinsic optimisation challenges, selecting optimal ranks is NP-hard in general [20]. Most methods resort to uniform rank assignment, which is suboptimal for LLMs where layers exhibit high heterogeneity. Current research into adaptive rank schemes aim to address this by allocating representational capacity where it is most needed, rather than enforcing a fixed global compression ratio [20]. Both CompactifAI [16] and Minima [19] find that deeper layers are significantly more compressible than earlier ones. Adaptive rank allocation based on layer sensitivity will therefore be essential for preserving reasoning capacity under aggressive compression.

Although LLM weights exhibit low-rank structure, aggressive truncation to very low bond dimensions inevitably discards information. Critically, this residual error is not random noise but retains recoverable structure that cannot be expressed by a purely low-rank factorisation. SATEN [21] addresses this by decomposing each weight matrix as the sum of a low-rank TT component and a sparse error term. On LlaMA-3.2-1B, this approach outperforms standalone TT and SVD-based compression, suggesting that future TN methods may need to complement low-rank assumptions with sparse corrections.

7 Conclusion

The current brute-force scaling paradigm has driven LLM performance to date, but faces a ceiling imposed by increasing training and deployment costs of huge models. Empirical evidence has also suggested low-rank structure and substantial redundancy in LLMs. This review has examined TNs as a mathematically principled alternative. By reshaping weight matrices into higher-order tensors, TNs expose multilinear correlations invisible to matrix-rank methods, with the bond dimensions controlling compression through an explicit structural prior. The compression rates achieved by this method confirm that TNs can effectively exploit the intrinsic low dimensional structure of LLM weight matrices, achieving ratios beyond conventional methods while preserving or improving performance.

A consistent finding is the structural asymmetry within transformers. FFN layers are smooth and highly compressible under simple decompositions, while attention mechanisms require factorisations that take into account inter-head geometry to avoid degradation. This implies that effective TN compression cannot be applied uniformly but must be tailored to the structural role of each component. Notably, TN compression can be complementary to existing techniques, as shown in CompactifAI [16] where the authors achieve large compression ratios by combining quantisation with TN decomposition.

However, the theoretical efficiency of TN decomposition is not realised with the current hardware which is highly optimised for dense matrix-matrix multiplication, limiting their widespread application. Progress depends on co-design: hardware-aware kernels minimising permutation overhead and ultimately accelerator architectures optimised for tensor contractions. If these engineering chal-

lenges are addressed, TN decomposition offers a viable path toward cheaper, faster models which retain the capabilities of their dense predecessors.

REFERENCES

[1] Ben Cottier, Robi Rahman, Loredana Fattorini, Nestor Maslej, Tamay Besiroglu, and David Owen. The rising costs of training frontier ai models, 2025. URL https://arxiv.org/abs/2405.21015.

[2] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020. URL https://arxiv.org/abs/2001.08361.

[3] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. Palm: Scaling language modeling with pathways, 2022. URL https://arxiv.org/abs/2204.02311.

[4] Andrey Gromov, Kushal Tirumala, Hassan Shapourian, Paolo Glorioso, and Daniel A. Roberts. The unreasonable ineffectiveness of the deeper layers, 2025. URL https://arxiv.org/abs/2403.17887.

[5] Román Orús. A practical introduction to tensor networks: Matrix product states and projected entangled pair states. *Annals of Physics*, 349:117–158, October 2014. ISSN 0003-4916. doi: 10.1016/j.aop.2014.06.013. URL http://dx.doi.org/10.1016/j.aop.2014.06.013.

[6] Prakhar Kaushik, Shravan Chaudhari, Ankit Vaidya, Rama Chellappa, and Alan Yuille. The universal weight subspace hypothesis, 2025. URL https://arxiv.org/abs/2512.05117.

[7] Pratyusha Sharma, Jordan T. Ash, and Dipendra Misra. The truth is in there: Improving reasoning in language models with layer-selective rank reduction, 2023. URL https://arxiv.org/abs/2312.13558.

[8] Flatiron Institute, Simons Foundation. The tensor network, 2025. URL https://tensornetwork.org/. Open-source review article.

[9] Alexander Novikov, Dmitry Podoprikhin, Anton Osokin, and Dmitry Vetrov. Tensorizing neural networks, 2015. URL https://arxiv.org/abs/1509.06569.

[10] Timur Garipov, Dmitry Podoprikhin, Alexander Novikov, and Dmitry Vetrov. Ultimate tensorization: compressing convolutional and fc layers alike, 2016. URL https://arxiv.org/abs/1611.03214.

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. URL https://arxiv.org/abs/1706.03762.

[12] Yuxuan Gu, Wuyang Zhou, Giorgos Iacovides, and Danilo Mandic. Tensorllm: Tensorising multi-head attention for enhanced reasoning and compression in llms, 2025. URL https://arxiv.org/abs/2501.15674.

[13] Xindian Ma, Peng Zhang, Shuai Zhang, Nan Duan, Yuexian Hou, Dawei Song, and Ming Zhou. A tensorized transformer for language modeling, 2019. URL https://arxiv.org/abs/1906.09777.

[14] Mingxue Xu, Yao Lei Xu, and Danilo P. Mandic. Tensorgpt: Efficient compression of large language models based on tensor-train decomposition, 2024. URL https://arxiv.org/abs/2307.00526.

[15] Yiran Luo, Het Patel, Yu Fu, Dawon Ahn, Jia Chen, Yue Dong, and Evangelos E. Papalexakis. Trawl: Tensor reduced and approximated weights for large language models, 2025. URL https://arxiv.org/abs/2406.17261.

[16] Andrei Tomut, Saeed S. Jahromi, Abhijoy Sarkar, Uygar Kurt, Sukhbinder Singh, Faysal Ishtiaq, Cesar Muñoz, Prabdeep Singh Bajaj, Ali Elborady, Gianni del Bimbo, Mehrazin Alizadeh, David Montero, Pablo Martin-Ramiro, Muhammad Ibrahim, Oussama Tahiri Alaoui, John Malcolm, Samuel Mugel, and Roman Orus. Compactifai: Extreme compression of large language models using quantum-inspired tensor networks, 2024. URL https://arxiv.org/abs/2401.14109.

[17] Safa Hamreras, Sukhbinder Singh, and Román Orús. Tensorization is a powerful but underexplored tool for compression and interpretability of neural networks, 2025. URL https://arxiv.org/abs/2505.20132.

[18] Sara Hooker. The hardware lottery, 2020. URL https://arxiv.org/abs/2009.06489.

[19] Sergii Kozyrev and Davyd Maiboroda. A practical tensor-network compression pipeline for production-scale large language models, 2026. URL https://arxiv.org/abs/2602.01613.

[20] Shiyi Luo, Mingshuo Liu, Yifeng Yu, Shangping Ren, and Yu Bai. An adaptive tensor-train decomposition approach for efficient deep neural network compression, 2025. URL https://arxiv.org/abs/2408.01534.

[21] Ryan Solgi, Kai Zhen, Rupak Vignesh Swaminathan, Nathan Susanj, Athanasios Mouchtaris, Siegfried Kunzmann, and Zheng Zhang. Saten: Sparse augmented tensor networks for post-training compression of large language models, 2025. URL https://arxiv.org/abs/2505.14871.